

“예로부터 나라의 인재는 성균에 모여 왔으니,
그대 머뭇이 우연이겠는가”

PySPH: a Python-based framework for smoothed particle hydrodynamics

PRABHU RAMACHANDRAN*†, ADITYA BHOSALE, KUNAL PURI, PAWAN NEGI, ABHINAV MUTA, A DINESH, DILEEP MENON, RAHUL GOVIND, SURAJ SANKA, AMAL S SEBASTIAN, ANANYO SEN, ROHAN KAUSHIK, ANSHUMAN KUMAR, VIKAS KURAPATI, MRINALGOUDA PATIL, DEEP TAVKER, PANKAJ PANDEY, CHANDRASHEKHAR KAUSHIK, ARKOPAL DUTT, and ARPIT AGARWAL

ACM (Association for Computing Machinery), Volume 1, issue 1, 39 pages, January 2020

May 5, 2025

Presenter: MinKyu Kang

mingu0215@g.skku.edu

- 연구 배경 및 문제 정의
- SPH 이론 및 논문 분석
- PySPH 구조 및 코드 분석
- 코드 실행 및 결과 분석
- 결론

“예로부터 나라의 인재는 성균에 모여 왔으니,
그대 머뭇이 우연이겠는가”

연구 배경 및 문제 정의

- 연구 배경 및 문제 정의:
 - 기존 SPH 패키지는 C/C++ 기반, Python 기반 부족
 - 낮은 생산성
 - 접근성 낮음
- 해결 방향:
 - Python 기반 SPH
 - PySPH만 Python + MPI + GPU 모두 지원

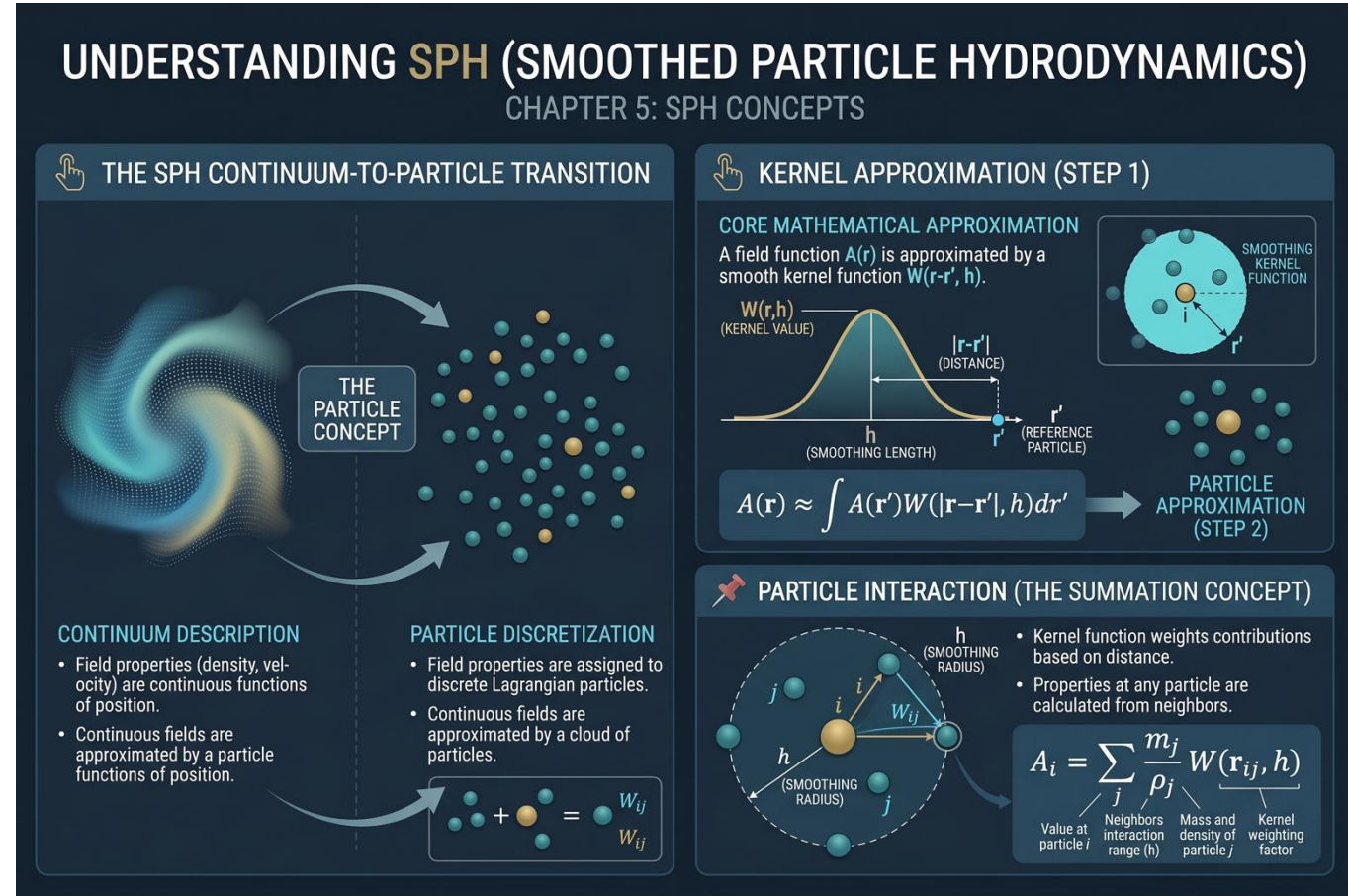
	MPI/GPU ¹	Language ²	NNPS ³	CI ⁴	Docs ⁵	Notes ⁶
AQUAGPU sph [5]	MPI & GPU	C++	Cell-grid	No	No	WCSPH
DualSPH ysics [3]	GPU	C++	Cell-grid	No	Yes	WCSPH, Elastic, DEM
GADGET4 [4]	MPI	C++	Octree	No	Yes	Compressible, Astrophysics
GIZMO [9]	MPI	C	Octree	No	Yes	Compressible, Astrophysics
GPUSPH [10]	MPI & GPU	C++	Cell-grid	No	Yes	WCSPH
PHANTOM [6]	MPI	FORTRAN	Octree	Yes	Yes	Compressible, Astrophysics
PySPH	MPI & GPU	Python	Cell-grid, Octree	Yes	Yes	WCSPH, ISPH, Compressible, Elastic, DEM
SPHERA [11]	No	FORTRAN	Cell-grid	No	No	WCSPH
SPHERAL++ [12]	MPI	C++/Python	Octree	No	No	WCSPH, Compressible, Astrophysics, Elastic
SPH inXsys [13]	No	C++	Cell-grid	No	Yes	WCSPH, Elastic
SPlisHSPlasH [7]	MPI & GPU	C++/Python	Cell-grid	No	No	WCSPH, ISPH, Elastic

Table 1. The summary of packages available for particle methods.

“예로부터 나라의 인재는 성균에 모여 왔으니,
그대 머뭇이 우연이겠는가”

SPH 이론 및 논문 분석

- 핵심 요소:
 - 연구 생산성 증가
 - 재사용성
 - 고성능 유지
- 목표 정의:
 - High-level Python
→ Low-level HPC 코드 자동 생성
 - CPU / GPU / MPI 지원
- SPH의 개념:
 - Kernel approximation
 - 연속체 → 입자 근사
 - Kernel 함수 사용

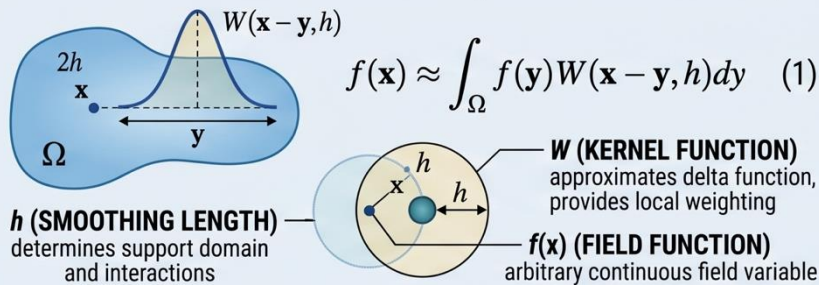


- Density equation
- Momentum equation
- 입자 간 상호작용 기반 계산

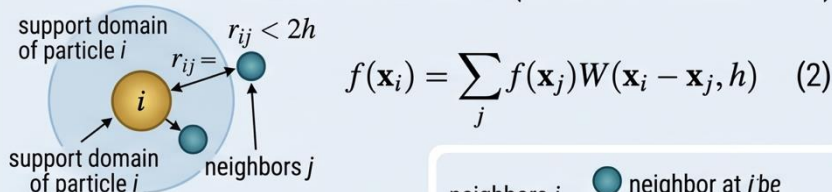
CORE SPH METHOD EQUATIONS: A LAGRANGIAN PARTICLE DISCRETIZATION

SECTION 1: DISCRETIZATION FOUNDATIONS

1. FIELD FUNCTION APPROXIMATION (KERNEL CONVOLUTION)

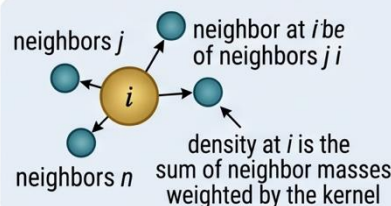


2. DISCRETE PARTICLE APPROXIMATION (SUMMATION OVER NEIGHBORS)



DENSITY EQUATION (3)

$$\rho(\mathbf{x}_i) = \sum_j m_j W(\mathbf{x}_i - \mathbf{x}_j, h)$$



SECTION 2: GOVERNING EQUATIONS IN SPH FORM

3. GOVERNING EQUATIONS & SPH DISCRETIZATION

LAGRANGIAN FLUID GOVERNING EQUATIONS

$$\frac{d\rho}{dt} = -\rho \nabla \cdot \mathbf{u} \quad \text{Eq 4} \quad \frac{d\mathbf{u}}{dt} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{g} \quad \text{Eq 5}$$

Material Derivative

Kinematic Viscosity

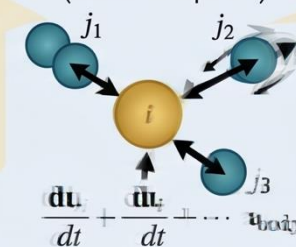
External Acceleration (e.g., gravity)

MOMENTUM EQUATION & PARTICLE INTERACTIONS (based on Eq 8 & 9)

PRESSURE ACCELERATION ($\mathbf{a}_p$)

$$\mathbf{a}_p = -\sum_j m_j \left(\frac{p_i}{\rho_i^2} + \frac{p_j}{\rho_j^2} \right) \nabla_i W_{ij}$$

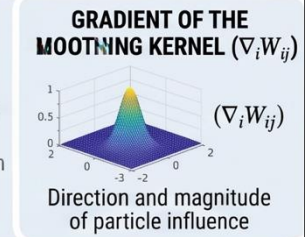
pressure gradient force, typically compressive



VISCOUS ACCELERATION ($\mathbf{a}_{visc}$)

$$\mathbf{a}_{visc} = \sum_j m_j \left[\frac{4\nu \mathbf{r}_{ij} \cdot \nabla_i W_{ij}}{(\rho_i + \rho_j)(|\mathbf{r}_{ij}|^2 + \eta h^2)} \right] \mathbf{u}_{ij}$$

viscous shear forces



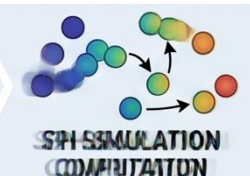
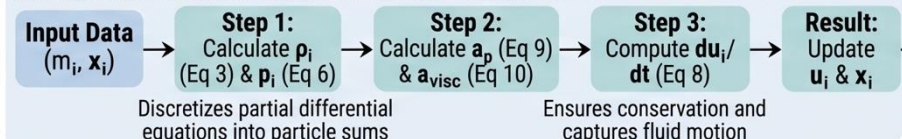
EQUATION OF STATE (6)

$$p = \frac{c_s^2 \rho_0}{\gamma} \left[\left(\frac{\rho}{\rho_0} \right)^\gamma - 1 \right]$$

Closes the system for weakly-compressible fluids

SECTION 3: SUMMARY & COMPUTATIONAL FLOW

SUMMARY OF PARTICLE-BASED CALCULATIONS



- NNPS 알고리즘
 - 연산량 줄이는 핵심 기술
 - Grid 방식
 - Octree 방식

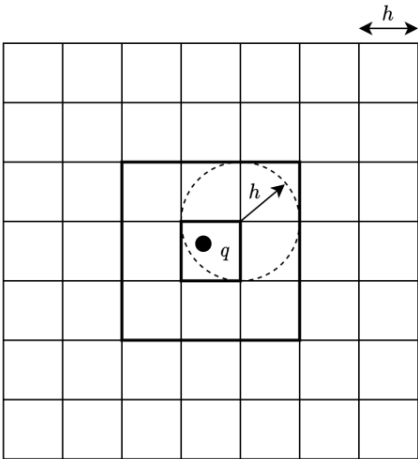
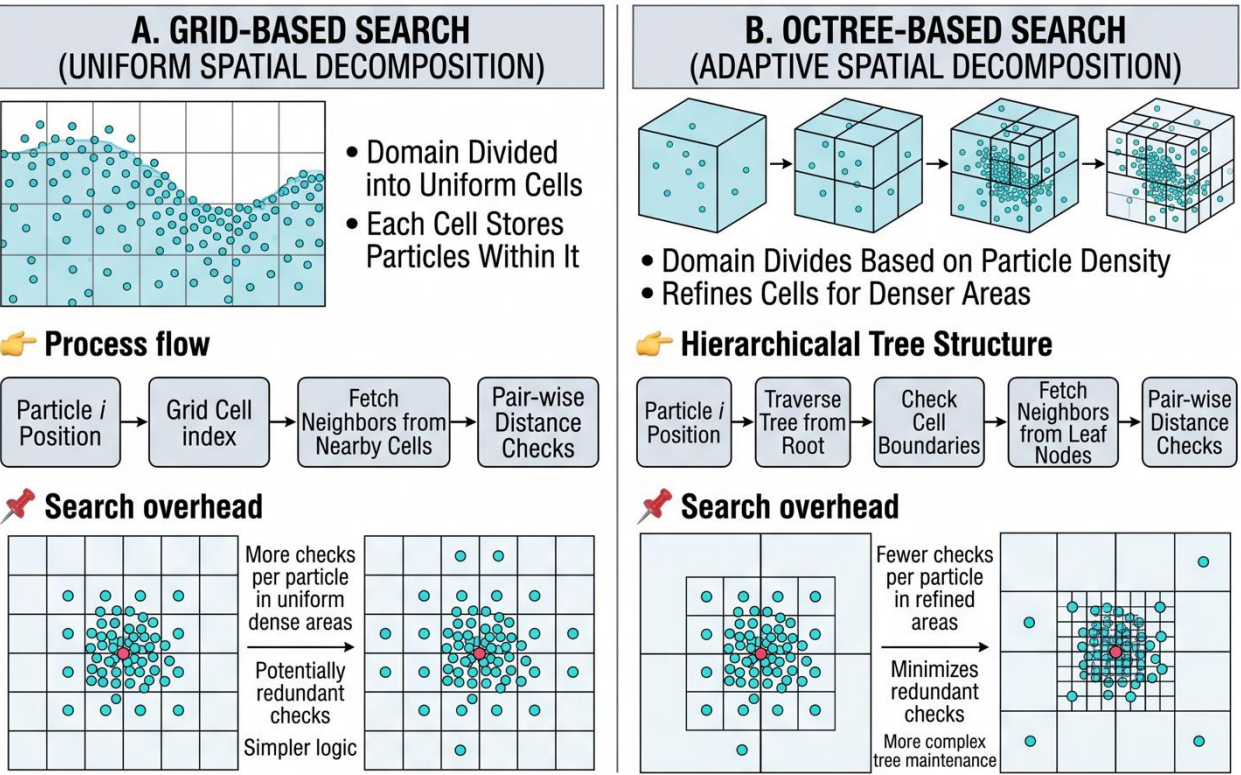


Fig. 1. Nearest neighbors of particle q with support radius h can be found in the 9 neighboring boxes of the cell that q belongs to.

7. NEIGHBOR SEARCH (NNPS): OPTIMIZING COMPUTATIONAL COST

CORE CONCEPT: ESSENTIAL TECHNOLOGY TO REDUCE COMPUTATIONAL COST



SUMMARY OF DIFFERENCES

Feature	Grid-Based	Octree-Based
Cell Size	Uniform	Adaptive
Density Handling	Fixed memory/time	Adaptive/variable
Traverse Complexity	$O(1)$ table lookup	$O(\log N)$ tree traversal
Memory Overhead	Lower, pre-allocated	Higher, tree nodes

BOTH METHODS SIGNIFICANTLY REDUCE $O(N^2)$ COMPUTATION FOR N-PARTICLE SYSTEMS

- 다양한 SPH 모델 지원
 - WCSPH
 - ISPH
 - Compressible SPH

8. SPH SCHEME (p.7) – DIVERSE SPH MODEL SUPPORT

ONE FRAMEWORK, DIVERSE PHYSICS

1. WCSPH (WEAKLY COMPRESSIBLE SPH)

EQUATION OF STATE (EOS)

- Explicit Density and Pressure ($p = f(\rho)$)

Eos: Highly Compressible for density maintenance

- Calculates density directly ($\rho_i = \sum_j m_j W_{ij}$)
- Pressure from EOS to keep fluid density near ρ_0
- Explicit Time Stepping
- Good for Free-Surface Flows (e.g., breaking waves)

2. ISPH (INCOMPRESSIBLE SPH)

PROJECTION METHOD

- Explicit intermediate velocity $\mathbf{u}^*$
- Poisson equation for pressure p

- Solves $\Delta p = f(\mathbf{u}^*)$

- Solves Pressure Poisson Equation (PPE)
- Enforces Incompressibility Condition ($\nabla \cdot \mathbf{u} = 0$, Constant ρ_i)
- Implicit Time Stepping (Pressure)
- Greater Stability for long simulations
- High computational cost per step

3. COMPRESSIBLE SPH

THERMODYNAMIC EQUATIONS

- Energy Equation (e)
- Density (ρ)
- Momentum ($\mathbf{u}$)

- Solves Energy Equation ($p = f(e, \rho)$)

- Coupled Energy & Momentum Solvers
- Enthalpy and Entropy formulations
- Shockwave capturing (e.g., Artificial Viscosity)
- Good for High-Speed and Thermal Flows

ONE FRAMEWORK, DIVERSE PHYSICS

May 2026

Department of Smart Factory Convergence 9/26

“예로부터 나라의 인재는 성균에 모여 왔으니,
그대 머뭇이 우연이겠는가”

PySPH 구조 및 코드 분석

1. Conda 환경 생성

```
minkyukang@minkyuui-noteubug ~/pysph main conda create -n pysph-test python=3.11 -y
Channels:
- defaults
Platform: osx-arm64
Collecting package metadata (repodata.json): done
Solving environment: done
```

```
minkyukang@minkyuui-noteubug ~/pysph main conda activate pysph-test
(pysph-test) pysph-test minkyukang@minkyuui-noteubug ~/pysph main
```

2. PySPH 설치

```
(pysph-test) pysph-test minkyukang@minkyuui-noteubug ~/pysph main pip install pysph
Collecting pysph
  Using cached pysph-1.0b2-cp311-cp311-macosx_12_0_arm64.whl
Collecting numpy (from pysph)
  Using cached numpy-2.4.4-cp311-cp311-macosx_14_0_arm64.whl.metadata (6.6 kB)
Collecting mako (from pysph)
  Using cached mako-1.3.12-py3-none-any.whl.metadata (2.9 kB)
Collecting cyarray (from pysph)
  Using cached cyarray-1.2-cp311-cp311-macosx_12_0_arm64.whl
Collecting compyle>=0.8 (from pysph)
  Using cached compyle-0.9.1-cp311-cp311-macosx_12_0_arm64.whl
Collecting Cython>=0.20 (from pysph)
  Using cached cython-3.2.4-cp311-cp311-macosx_11_0_arm64.whl.metadata (7.5 kB)
Requirement already satisfied: setuptools>=42.0.0 in /opt/anaconda3/envs/pysph-test/lib/python3.11/site-packages (from pysph) (82.0.1)
```

3. PySPH 실행

```
(pysph-test) pysph-test minkyukang@minkyuui-noteubug ~/pysph main pysph run elliptical_drop
Running example pysph.examples.elliptical_drop.
```

Information for example: pysph.examples.elliptical_drop
Evolution of a circular patch of incompressible fluid. (60 seconds)

See J. J. Monaghan "Simulating Free Surface Flows with SPH", JCP, 1994, 100, pp 399 - 406

An initially circular patch of fluid is subjected to a velocity profile that causes it to deform into an ellipse. Incompressibility causes the initially circular patch to deform into an ellipse such that the area is conserved. An analytical solution for the locus of the patch is available (exact_solution)

This is a standard test for the formulations for the incompressible SPH equations.

Elliptical drop :: 5025 particles
Effective viscosity: $\rho \cdot \alpha \cdot h \cdot c / 8 = 0.5687500000000001$
Generating output in /Users/minkyukang/pysph/elliptical_drop_output

No of particles:
fluid: 5025

Setup took: 0.06299 secs
100%|██████████| 1.1kit | 7.6e-03s [00:26.6<00:0.0 | 0.024s/it]
Run took: 26.64244 secs
Post processing requires matplotlib.

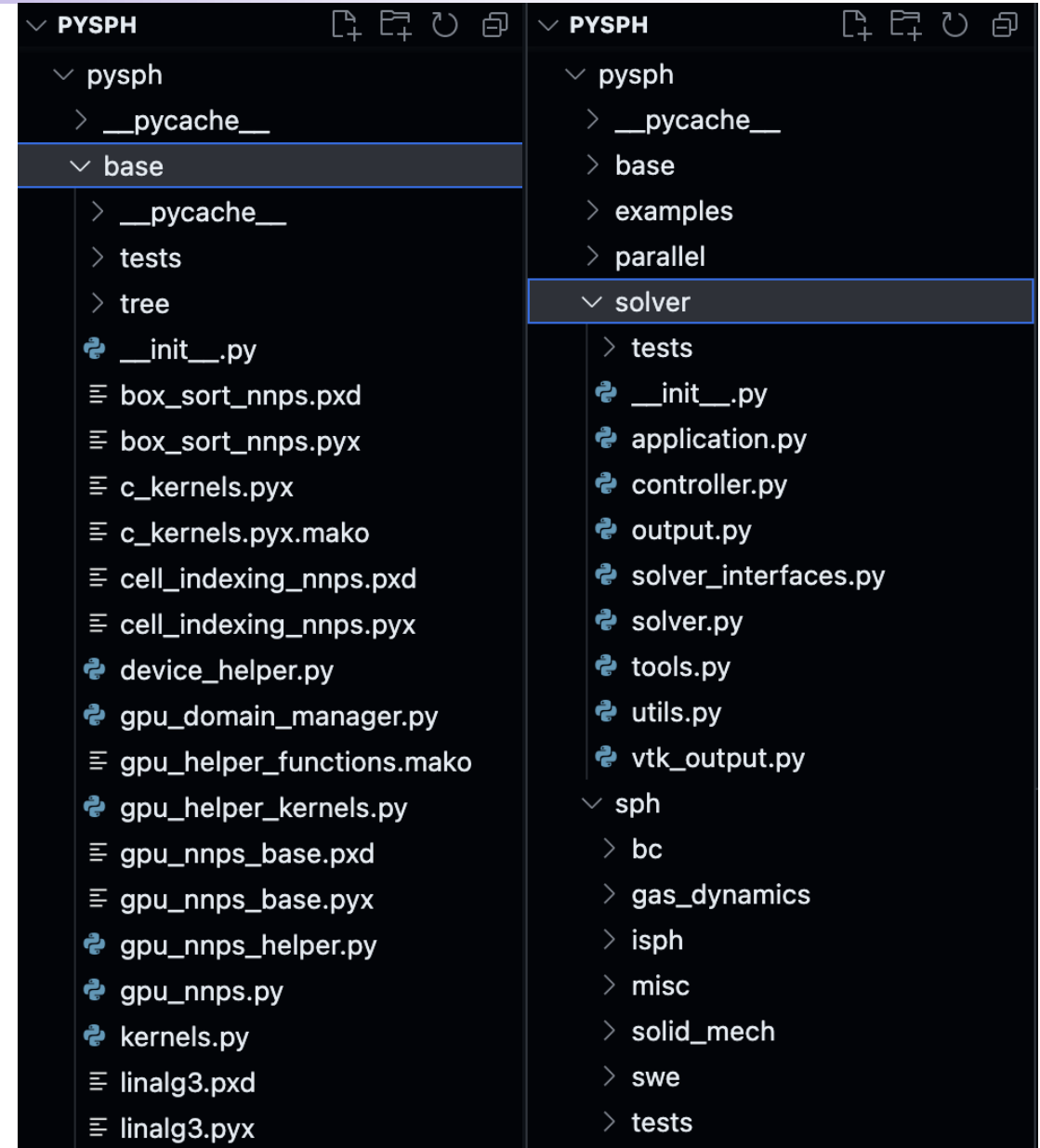
4. GitHub 코드 확보(Git Clone)

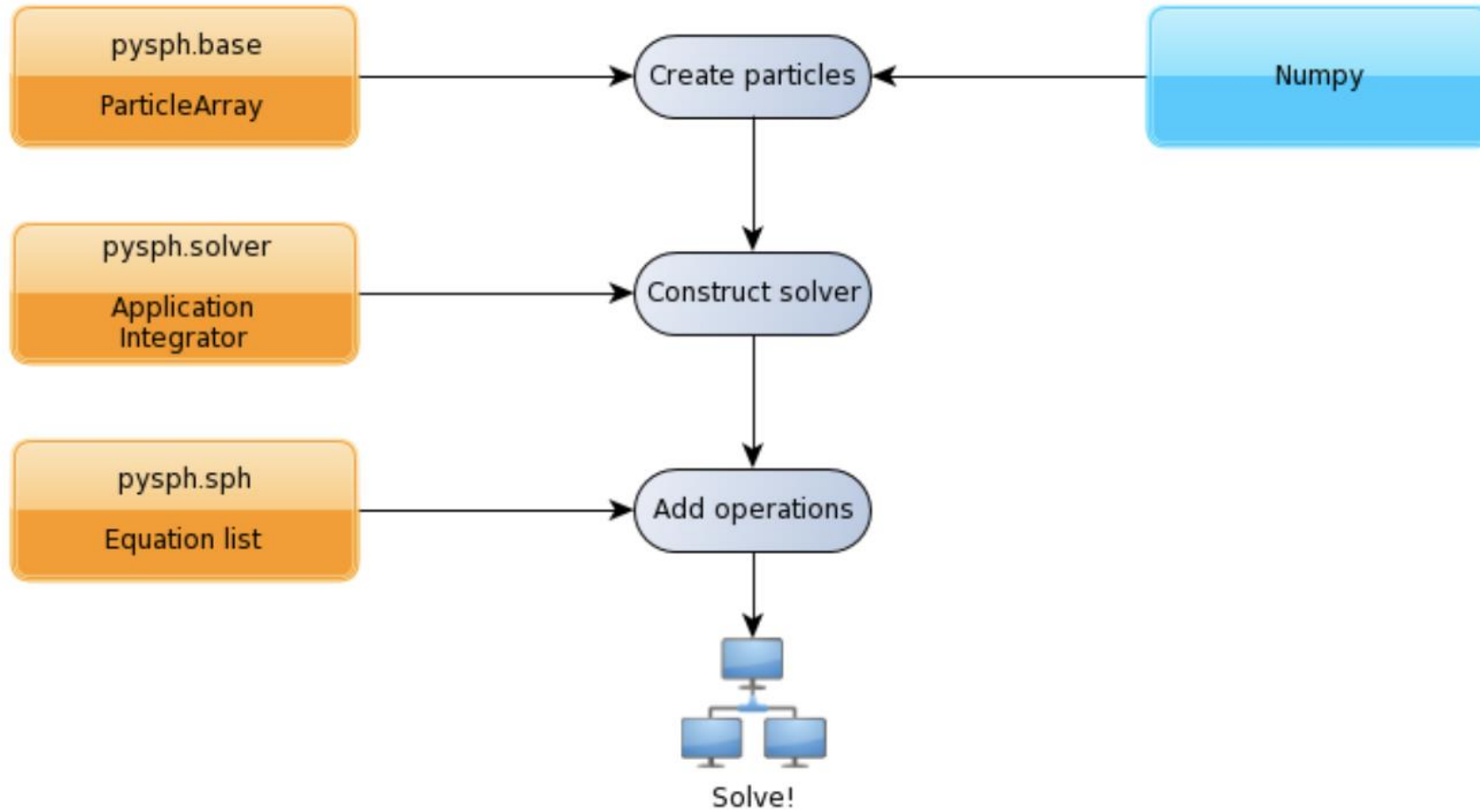
```
(pysph-test) pysph-test minkyukang@minkyuui-noteubug ~/pysph main git clone https://github.com/pypr/pysph.git
fatal: destination path 'pysph' already exists and is not an empty directory.
```

타겟논문 오픈소스 링크: <https://github.com/pypr/pysph>

PySPH 모듈 구조(오픈소스)

- pysph.base → 데이터(base 모듈)
 - ParticleArray
 - 입자 정보 관리
 - 속성 저장 (x, y, rho 등)
- pysph.sph → 물리(sph 모듈)
 - Equation 클래스
 - 물리 방정식 정의
 - Python → C 코드 변환
- pysph.solver → 실행(solver 구조)
 - Application → Solver → Integrator
 - create_particles()
 - create_equations()
 - create_solver()
- pysph.examples





Equation 구조와 Integrator

- Equation 구조
 - 모든 물리 계산은 loop에서 발생
 - initialize → 초기화
 - loop → 입자 간 계산
- Integrator
 - Euler 방식
 - 시간 적분
 - 위치/속도 업데이트

```

1 Integrator related modules
2 =====
3
4 .. automodule:: pysph.sph.integrator
5     :members:
6     :undoc-members:
7
8 .. automodule:: pysph.sph.integrator_step
9     :members:
10    :undoc-members:
11
12 .. autoclass:: pysph.sph.gas_dynamics.magma2.TVDRK2Integrator
13     :special-members:
14
15 .. autoclass:: pysph.sph.gas_dynamics.magma2.TVDRK2IntegratorWithRecycling
16     :special-members:
17
18 .. autoclass:: pysph.sph.gas_dynamics.magma2.TVDRK2Step
19     :special-members:
20

```

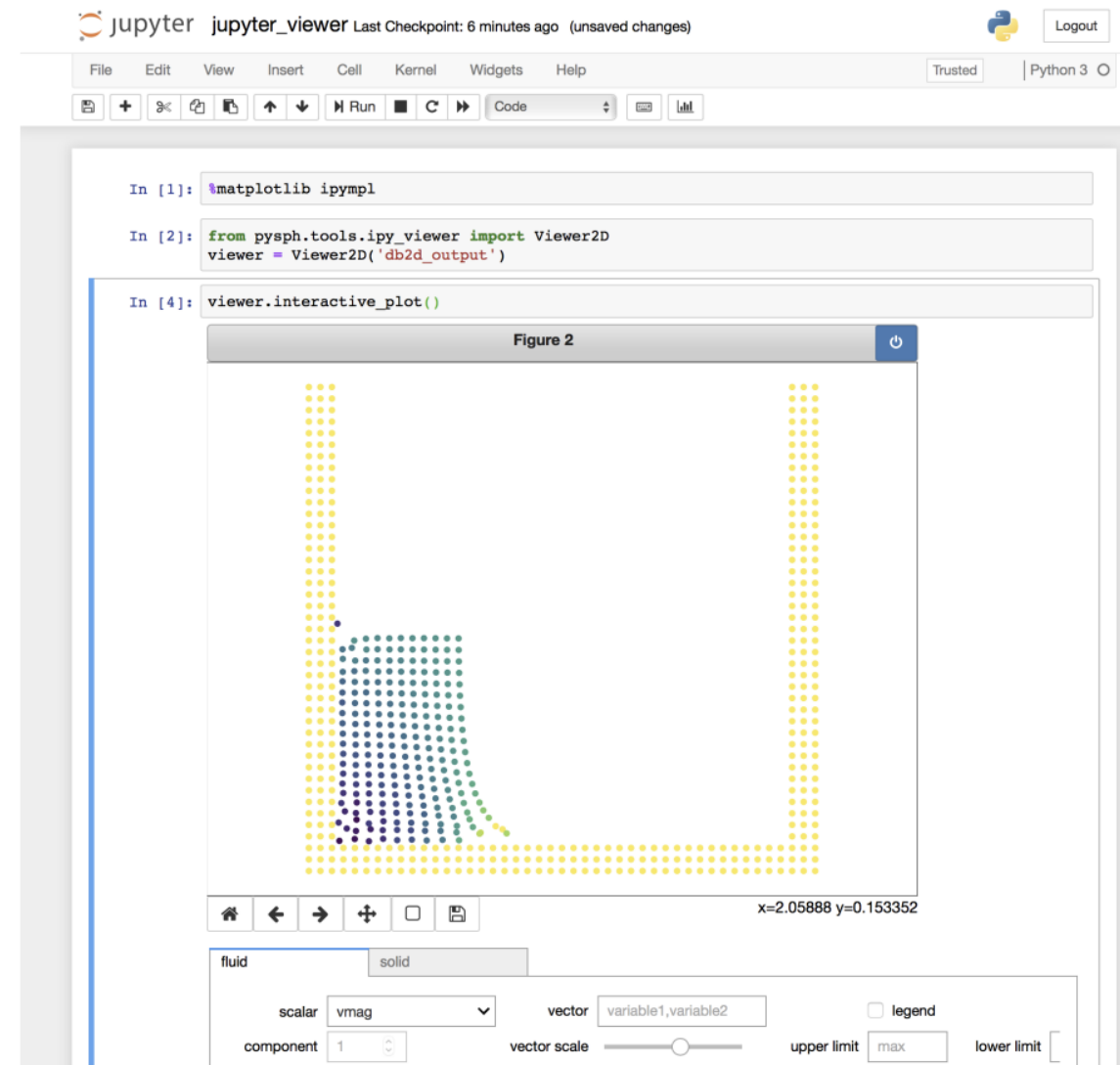
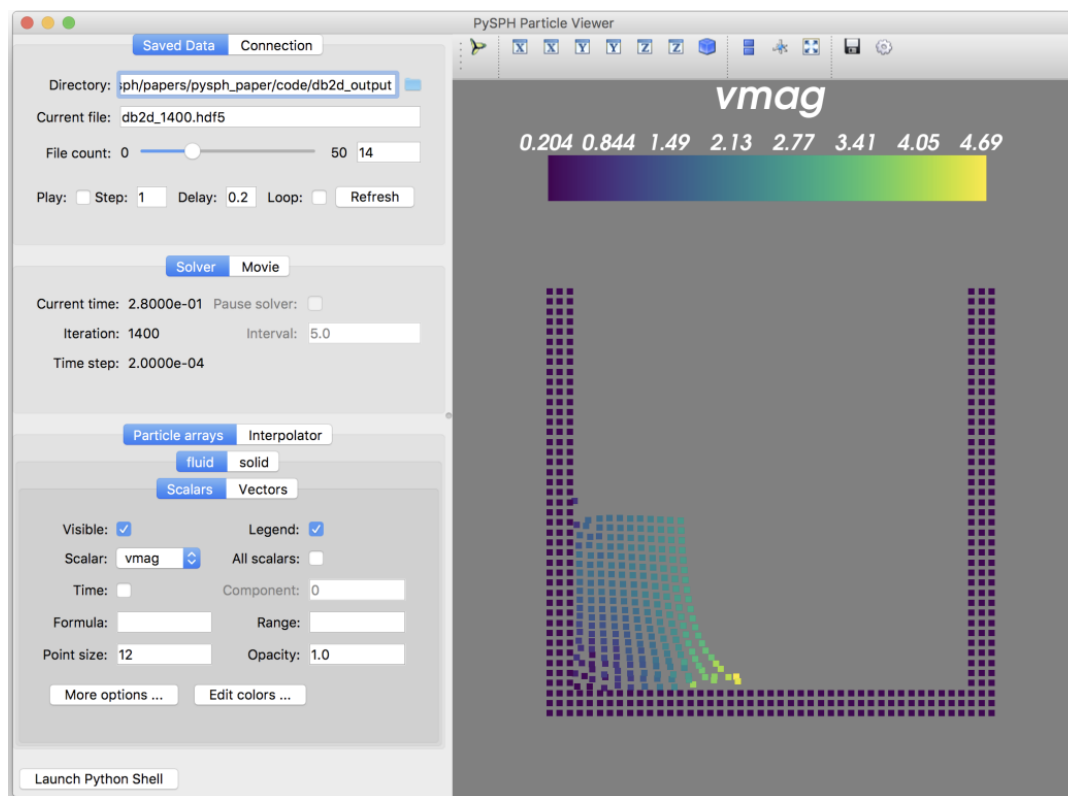
```

1  .. _writing_equations:
2
3  =====
4  Writing equations
5  =====
6
7  This document puts together all the essential information on how to write
8  equations. We assume that you have already read the section
9  :ref:`design_overview`. Some information is repeated from there as well.
10
11  The PySPH equations are written in a very restricted way. The reason for this
12  is that if you do follow the suggestions and the conventions below you will
13  benefit from:
14
15  - a high-performance serial implementation.
16  - support for using your equations with OpenMP.
17  - support for running on a GPU.
18
19  These are the main motivations for the severe restrictions we impose when you
20  write your equations.
21
22  Overview
23  =====
24
25  PySPH takes the equations you write and converts them on the fly to a
26  high-performance implementation suitable for the particular backend you
27  request.
28
29  .. py:currentmodule:: pysph.sph.equation
30
31  It is important to understand the overall structure of how the equations are
32  used when the high-performance code is generated. Let us look at the different
33  methods of a typical :py:class:`Equation` subclass::

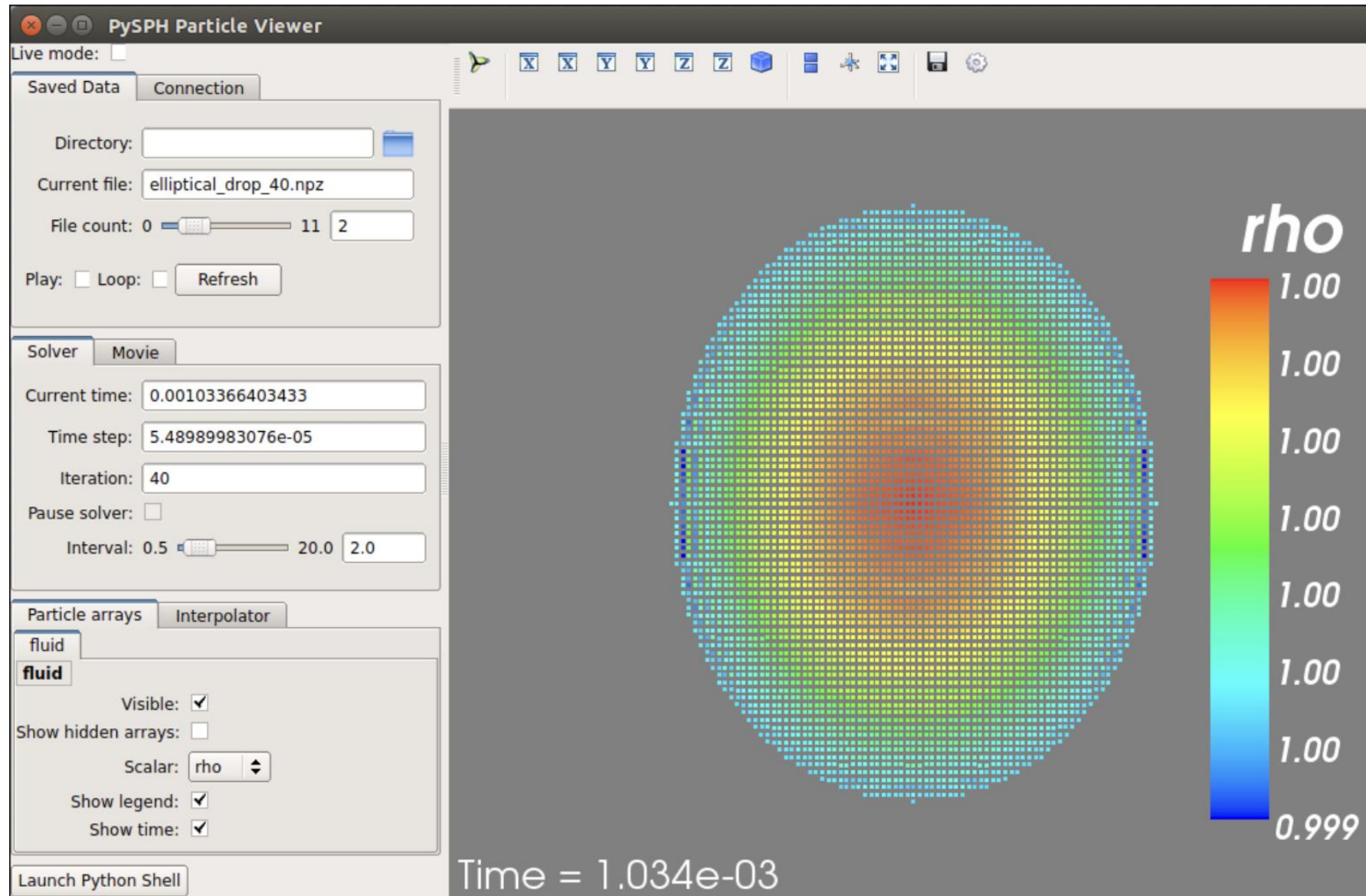
```

Code Generation & Implementation

- Python → Cython → 실행
- Python 코드지만 실제로는 HPC 코드
 - mako template 사용
 - 자동 코드 생성



Code Generation & Implementation

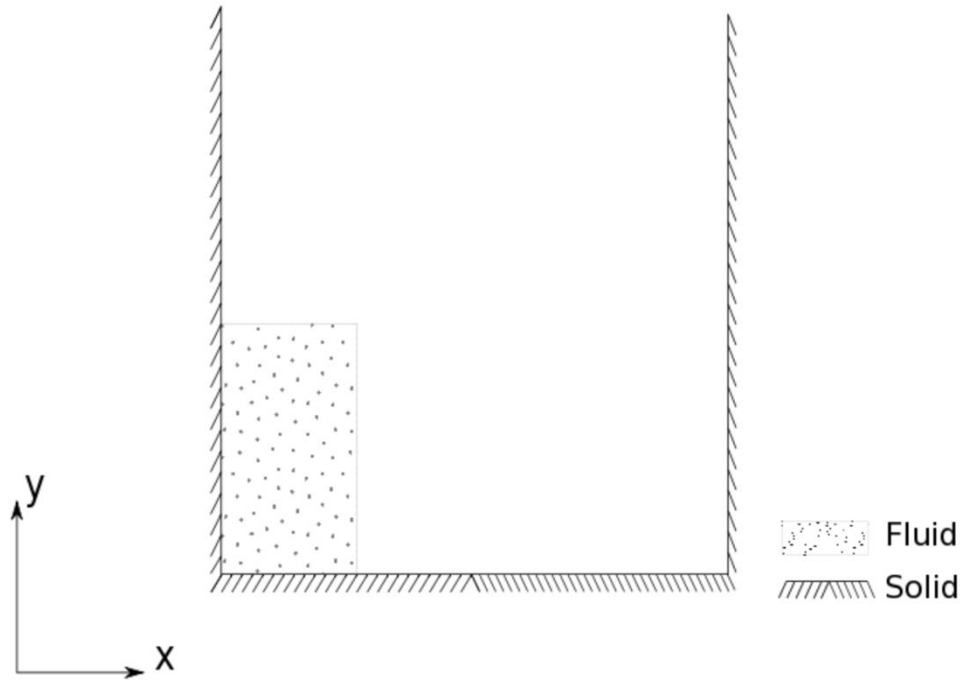


“예로부터 나라의 인재는 성균에 모여 왔으니,
그대 머뭇이 우연이겠는가”

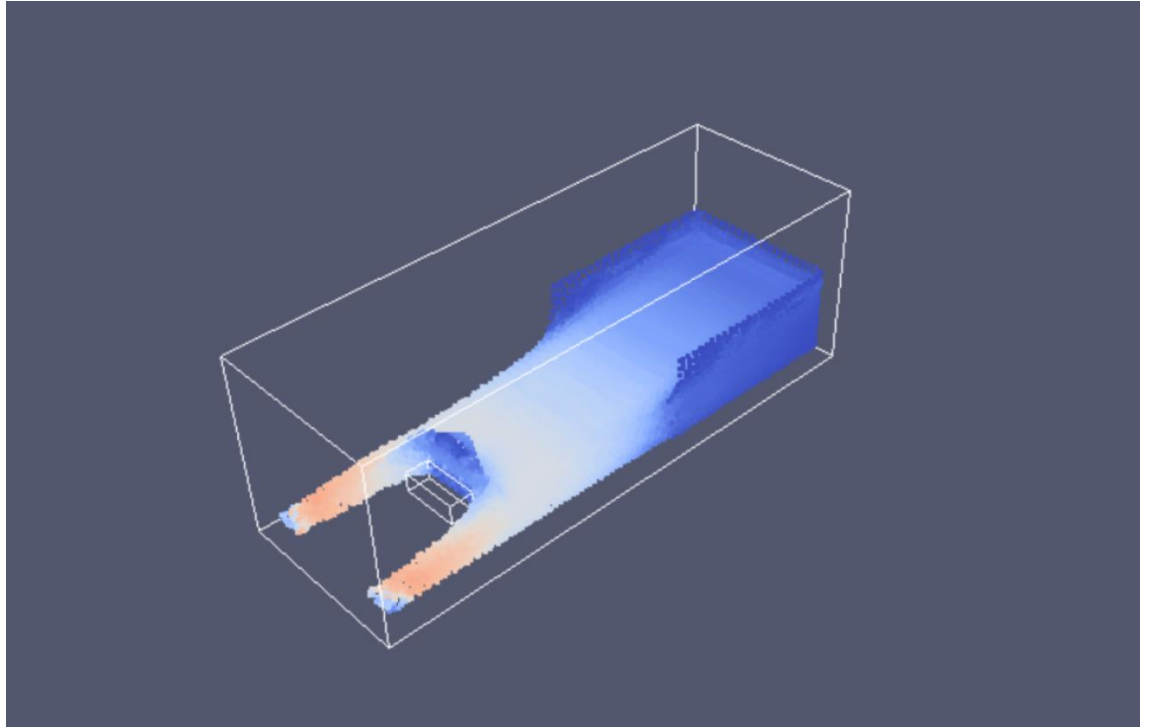
결론

- PySPH 기반 SPH 유체 시뮬레이션 수행
- 입자 기반 모델로 실제 유체 거동을 효과적으로 재현
 - 주요 테스트:
 - Dam-break (유체 붕괴)
 - Lid-driven cavity (순환 유동)
 - Particle partition (병렬 구조)
- 속도 분포 시각화
- 입자 기반 유체 흐름 확인 가능

dam-break-schematic



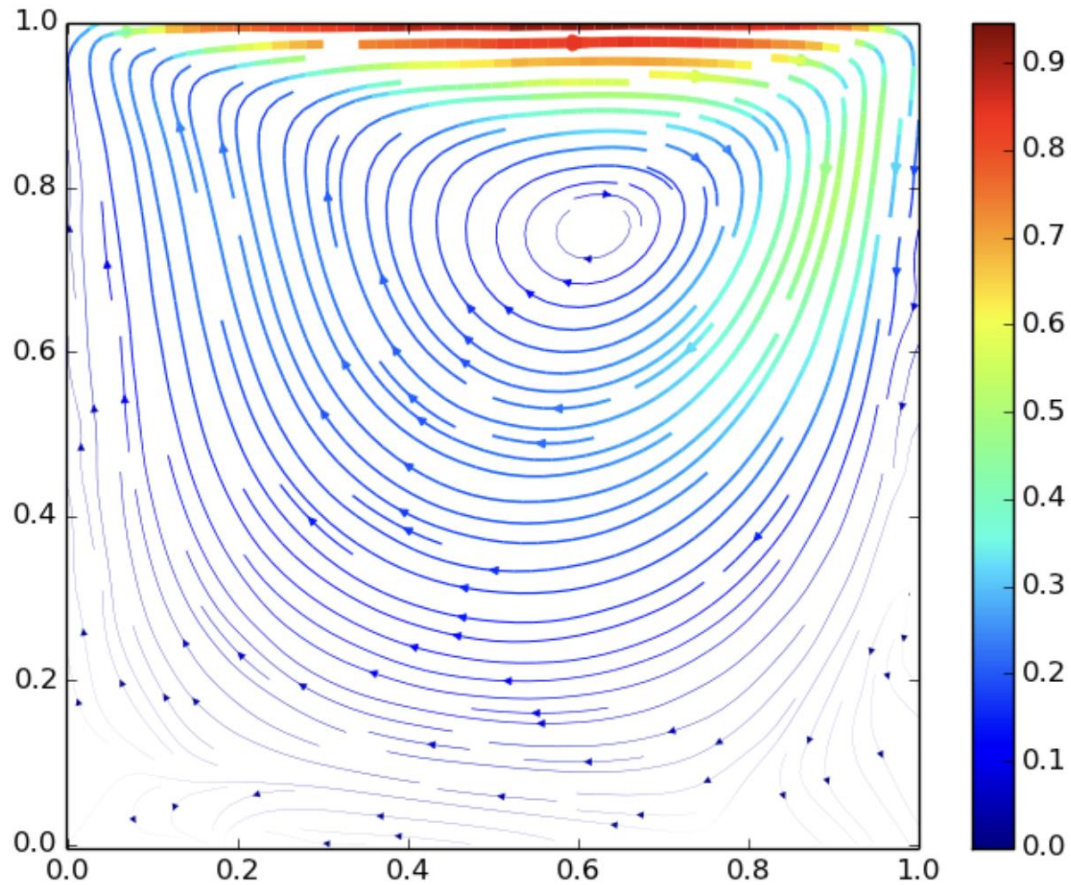
db3d



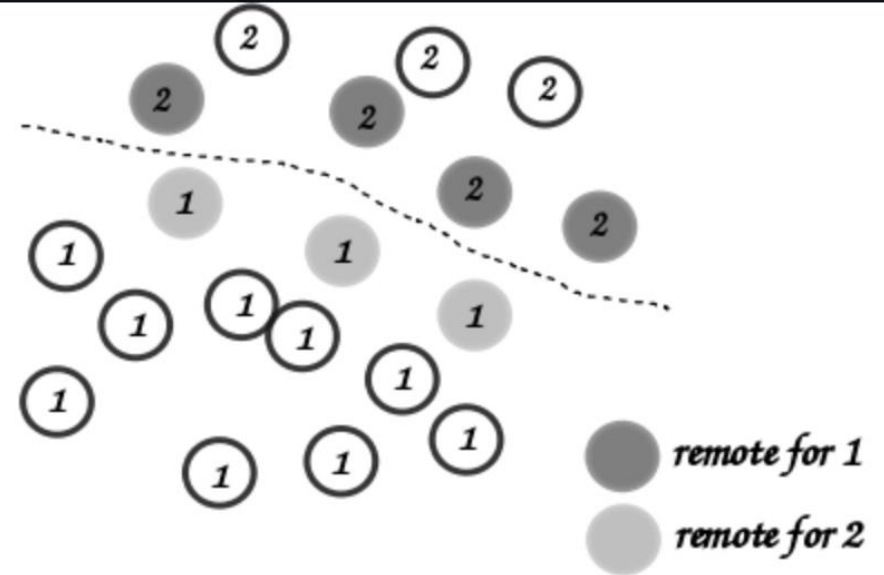
결과 시각화(Dam-break 분석)

- 초기: 정지된 유체 블록 → 중력에 의해 붕괴
- 진행: 수평 확산 + 벽 충돌 → 반사 및 난류 발생
- 속도:
 - 전면부 최대
 - 후방 및 벽면 근처 gradient 형성
- 자유 표면 흐름 자연스럽게 표현
- 압력 + 점성 항이 정상적으로 작동
- 입자 상호작용만으로 물리적 유동 재현

ldc-streamlines



local-remote-particles

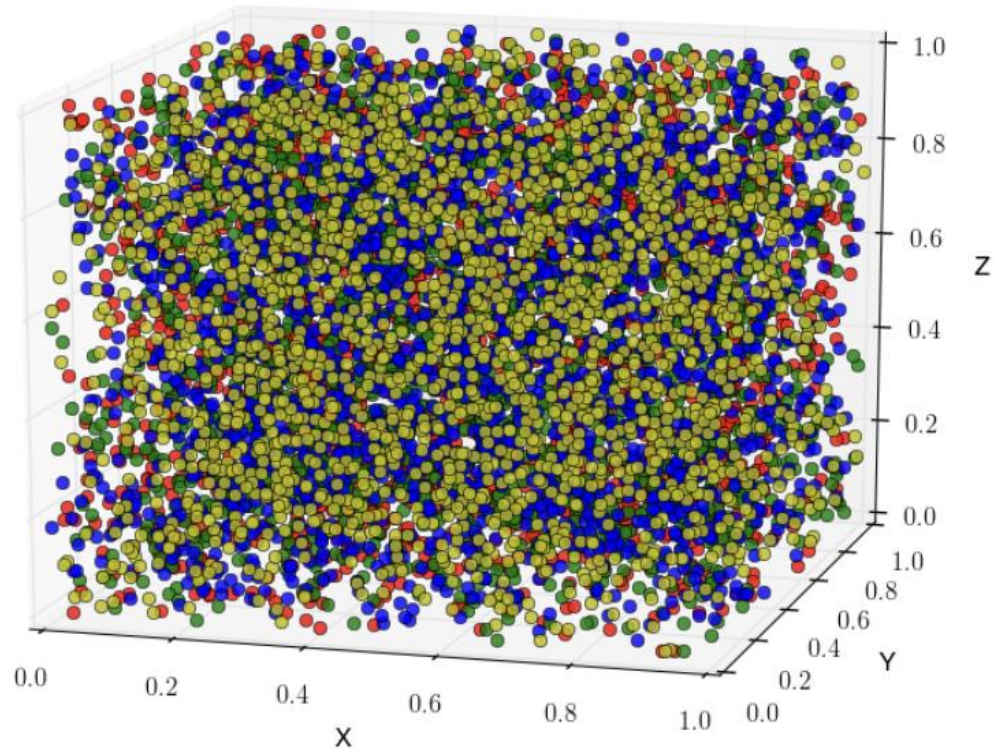


결과 시각화 (Lid-driven cavity 분석)

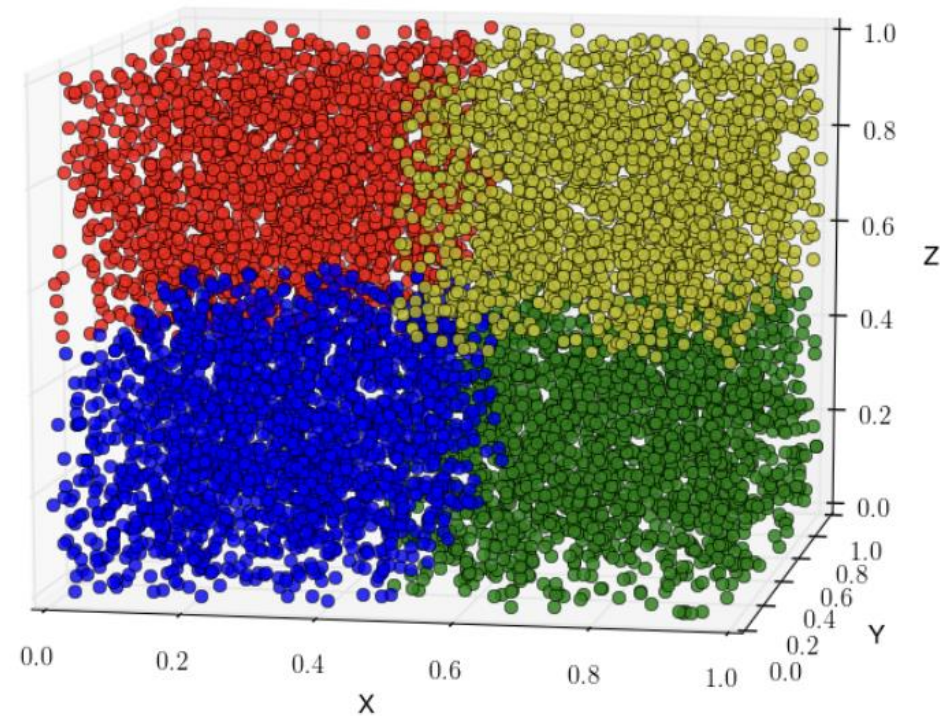
- 상단 경계 이동 → 내부 순환 유동 발생
- 결과:
 - 중심 vortex 형성
 - 벽 근처 2차 vortex 발생
- 복잡한 내부 유동 구조까지 정확히 표현 가능

point-partition

Initial Distribution



Final Distribution



- 구조·성능 및 물리적 타당성
 - Local / Remote particle 구조
 - 공간 기반 입자 분할
 - MPI 기반 병렬 처리
 - 대규모 시뮬레이션 확장 가능
 - 계산 효율 + 확장성 확보
- PySPH 구조와 결과 연결
 - ParticleArray → 상태 관리
 - Equation → 물리 계산
 - Integrator → 시간 변화
 - 이 구조가 결합되어 실제 유체 흐름 생성

- PySPH는 Python 기반으로 고성능 유체 시뮬레이션을 구현하며, SPH 이론을 실제 물리 현상으로 재현하는 데 효과적인 프레임워크
- 물리적 타당성
 - 질량 보존
 - 운동량 보존
 - 자유 표면 처리
 - 물리적으로 일관된 결과 도출
- 기존 CFD 대비 특징

구분	SPH (PySPH)	Grid 기반
구조	Meshless	Mesh 필요
자유표면	자연스러움	어려움
병렬화	쉬움	복잡

“예로부터 나라의 인재는 성균에 모여 왔으니,
그대 머뭇이 우연이겠는가”

감사합니다.